RUL Prediction Project — Final Notes

Dataset: NASA C-MAPSS FD001 **Task:** Remaining Useful Life (RUL) Regression

1. Project Objective

Predict the Remaining Useful Life of an aircraft engine from its operating-cycle sensor readings. RUL represents the number of useful operating cycles remaining before failure.

2. Complete Pipeline

Raw sensor data → RUL calculation → sensor correlation analysis → relevant sensor selection → temporal feature engineering → missing-value handling → engine-wise train/validation split → XGBoost regression → model comparison → unseen test prediction → evaluation → error analysis → model saving.

3. RUL Calculation

For training data, RUL was calculated as:

```
RUL = maximum cycle of that engine – current cycle
```

Example: if an engine's maximum observed cycle is 192 and the current cycle is 10, RUL = 182.

4. Sensor/Correlation Analysis

Correlation with RUL was used to identify sensors whose behavior was related to degradation. Examples from the analysis: sensor_11 ≈ –0.696, sensor_4 ≈ –0.679, sensor_12 ≈ 0.672, sensor_7 ≈ 0.657, sensor_15 ≈ –0.643 and sensor_21 ≈ 0.636. Engine-wise mean/std correlation was also examined for stability.

5. Feature Engineering

The engineered dataset contained 68 columns including identifiers/target; the model input contained 65 features. For selected sensors, the following temporal features were created:

• **Delta:** current sensor value – previous-cycle sensor value.

• **Rolling mean (5):** average over the recent 5 cycles.

• **Rolling mean (10):** average over the recent 10 cycles.

• **Rolling standard deviation (10):** recent 10-cycle variability.

6. Missing Values

The first cycle of each engine naturally produced missing delta/rolling-statistic values. The engineered dataset showed 2,000 missing values across the reported delta/std columns. These were handled as part of the preprocessing pipeline before model evaluation.

7. Train/Validation Split

Training: 80 engines, 16,561 rows, 65 model features.
Validation: 20 engines, 4,070 rows, 65 model features.

The split was done at the engine level rather than randomly by row. This avoids putting highly correlated cycles from the same engine into both training and validation, reducing leakage and giving a more meaningful validation estimate.

8. Model

XGBoost regression was used because the final data is tabular and contains nonlinear relationships and interactions between sensor behavior, cycle information and RUL.

9. Model Comparison

Model	MAE	RMSE	R^2
Model_1	23.497	31.006	0.777
Model_2	23.576	30.966	0.778
Model_3	23.492	31.133	0.775

Model_2 had the highest validation R^2 and the lowest validation RMSE among these three, so it was treated as the selected model.

10. Final Performance

Metric	Validation	Final Test
MAE	23.58	19.71
RMSE	30.97	27.68
R^2	0.778	0.556

11. How to Explain the Metrics

MAE: average absolute prediction error in cycles. Final test MAE 19.71 means predictions were, on average, about 19.7 cycles away from actual RUL.
RMSE: penalizes large errors more strongly than MAE. Final test RMSE was 27.68.
R^2: measures how much variation in RUL is explained by the model. Final test R^2 was 0.556.

12. Feature Importance

Top features included cycle (0.176), sensor_4_roll_mean_5 (0.091), sensor_11_roll_mean_5 (0.088), sensor_15_roll_mean_10 (0.075), followed by several other rolling sensor features. The importance results indicate that cycle and recent sensor trends were particularly useful to the XGBoost model.

13. Error Analysis

The final test-point error analysis showed a positive mean error of about +11.13 cycles and median error of about +5.87, indicating an overall tendency toward overprediction. Some engines had much larger errors. Examples: engine 67 actual 77 vs predicted 169.99 (error +92.99), engine 1 actual 112 vs predicted 194.24 (+82.24), and engine 78 actual 107 vs predicted 177.84 (+70.84).

The validation R^2 (0.778) being higher than test R^2 (0.556) also indicates that generalization to unseen test engines is imperfect. This is an important limitation to mention rather than claiming the model is perfect.

14. Important Interview Answers

Q: Why XGBoost?

A: The engineered dataset is tabular, and XGBoost handles nonlinear relationships and feature interactions well.

Q: Why feature engineering?

A: A single sensor reading does not fully represent degradation. Delta and rolling statistics capture recent trends and variability.

Q: Why engine-wise split?

A: Cycles from the same engine are correlated. A random row split could leak engine-specific patterns into validation.

Q: What does MAE 19.71 mean?

A: The model's test predictions differ from actual RUL by about 19.7 cycles on average.

Q: Why is test R^2 lower?

A: The test contains unseen engines with different degradation patterns, and error analysis shows several large prediction errors.

15. Files Saved

The notebook successfully created and verified these files:

• final_rul_xgboost_model.pkl

• rul_feature_names.pkl

• final_rul_predictions.csv

Model file: the trained XGBoost model was successfully saved as final_rul_xgboost_model.pkl.
Feature file: rul_feature_names.pkl contains the 65 feature names used by the model.
Prediction file: final_rul_predictions.csv contains 100 test-engine predictions with actual RUL, predicted RUL, error and absolute error.

16. 30-Second Project Pitch

“I built a predictive-maintenance model to estimate the Remaining Useful Life of aircraft engines using NASA C-MAPSS FD001. I calculated cycle-level RUL, performed sensor correlation analysis, and engineered temporal features such as deltas, rolling means and rolling standard deviations. I used an engine-wise train-validation split to reduce leakage and trained XGBoost regression models. The selected model achieved 23.58 MAE and 0.778 R^2 on validation, and 19.71 MAE, 27.68 RMSE and 0.556 R^2 on the final unseen test set. I also performed feature-importance and error analysis to understand where the model performed well and where it struggled.”